

Nazwa kwalifikacji: **Projektowanie, programowanie i testowanie aplikacji**
Oznaczenie kwalifikacji: **INF.04**
Numer zadania: **04**

Wypełnia zdający

Numer Identyfikacyjny zdającego*

--	--	--	--	--	--	--	--

Miejsce na numer ośrodka

Czas trwania egzaminu: **180** minut.

INF.04-KG-26.04

EGZAMIN ZAWODOWY

Rok 2026

CZĘŚĆ PRAKTYCZNA

**PODSTAWA PROGRAMOWA
2019**

Instrukcja dla zdającego

1. Na pierwszej stronie arkusza egzaminacyjnego wpisz w oznaczonym miejscu swój numer Identyfikacyjny.
2. Na KARCIE OCENY wpisz swoje dane:
 - swój numer Identyfikacyjny,
 - oznaczenie kwalifikacji,
 - numer zadania,
 - numer stanowiska.
3. Sprawdź, czy arkusz egzaminacyjny zawiera 6 stron i nie zawiera błędów. Ewentualny brak stron lub inne usterki zgłoś przez podniesienie ręki przewodniczącemu zespołu nadzorującego.
4. Zapoznaj się z treścią zadania oraz stanowiskiem egzaminacyjnym. Masz na to 10 minut. Czas ten nie jest wliczany do czasu trwania egzaminu.
5. Czas rozpoczęcia i zakończenia pracy zapisze w widocznym miejscu przewodniczący zespołu nadzorującego.
6. Wykonaj samodzielnie zadanie egzaminacyjne. Przestrzegaj zasad bezpieczeństwa i organizacji pracy.
7. Po zakończeniu wykonania zadania pozostaw arkusz egzaminacyjny z rezultatami oraz KARTĘ OCENY na swoim stanowisku lub w miejscu wskazanym przez przewodniczącego zespołu nadzorującego.
8. Po uzyskaniu zgody zespołu nadzorującego możesz opuścić salę/miejsce przeprowadzania egzaminu.

Powodzenia!

* w przypadku braku numeru ID – seria i numer paszportu lub innego dokumentu potwierdzającego tożsamość

Zadanie egzaminacyjne

UWAGA: katalog z rezultatami pracy oraz płytę należy opisać numerem zdającego, którym został podpisany arkusz. Dalej w zadaniu numer ten jest nazwany numerem zdającego.

Wykonaj aplikację konsolową oraz webową według wskazań. Udokumentuj obie aplikacje zrzutami ekranu. Wykorzystaj konto **Egzamin** bez hasła.

Utwórz folder i nazwij go numerem zdającego. W folderze utwórz podfoldery: *konsola*, *webowa*, *testy*. Po wykonaniu każdej aplikacji, jej pełny kod (cały folder projektu) spakuj do archiwum. Następnie pozostaw w podfolderze jedynie pliki źródłowe, których treść była modyfikowana, plik wykonywalny, jeśli jest to możliwe oraz spakowane archiwum.

Część I. Aplikacja konsolowa

Za pomocą narzędzi do tworzenia aplikacji konsolowych zaimplementuj program służący do odnajdywania odpowiedniej choinki metodą wyszukiwania binarnego.

Założenia programu

- Program wykonywany w konsoli.
- Język programowania zgodny z zainstalowanym na stanowisku egzaminacyjnym, jeden z: C++, C#, Java, Python.
- Podejście obiektowe lub strukturalne.
- Program wykorzystuje algorytm wyszukiwania binarnego do znalezienia odpowiedniej choinki.
- Program powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.

Opis programu

Program wyszukuje choinkę o zadanej rozmiarze. Na początku użytkownik wprowadza liczbę całkowitą dodatnią n , oznaczającą liczbę dostępnych choinek o rozmiarach od 1 do n .

Następnie program zadaje pytania w postaci: "Czy to choinka o rozmiarze: $\langle lb \rangle$?", gdzie $\langle lb \rangle$ to pytana liczba.

Użytkownik odpowiada jednym z symboli:

- > gdy szukana choinka jest większa od podanej,
- < gdy szukana choinka jest mniejsza od podanej,
- = gdy podana choinka jest właściwa.

W przypadku odpowiedzi > lub < program kontynuuje wyszukiwanie.

Po otrzymaniu odpowiedzi = program wyświetla komunikat

"Hurrrrrrra! Znalazłem twoją choinkę!" i kończy działanie.

Przykłady interakcji programu znajdują się na obrazach 1a i 1b.

Założenia dotyczące kodu

- Wykorzystuje algorytm wyszukiwania binarnego
- Odpowiedź użytkownika znajduje się na końcu aktualnej linii z pytaniem
- Nie jest wymagana walidacja danych – zakłada się, że wprowadzono go bez błędów

```
Podaj liczbę choinek: 5
Czy to choinka o rozmiarze: 3 ? <
Czy to choinka o rozmiarze: 1 ? >
Czy to choinka o rozmiarze: 2 ? =
Hurrrrrrra! Znalazłem twoją choinkę!
```

Obraz 1a. Interakcja programu dla choinki równej 2

```
Podaj liczbę choinek: 100
Czy to choinka o rozmiarze: 50 ? >
Czy to choinka o rozmiarze: 75 ? <
Czy to choinka o rozmiarze: 62 ? >
Czy to choinka o rozmiarze: 68 ? >
Czy to choinka o rozmiarze: 71 ? >
Czy to choinka o rozmiarze: 73 ? =
Hurrrrrrra! Znalazłem twoją choinkę!
```

Obraz 1b. interakcja programu dla choinki równej 73

Po uruchomieniu programu wykonaj zrzuty ekranu dokumentujące interakcje programu dla różnych wartości roku. Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym. Liczba zrzutów ekranu powinna odpowiadać wszystkim możliwym interakcjom użytkownika z programem. Zrzuty ekranu zapisz w folderze konsolowa pod nazwami *konsola1.png*, *konsola2.png*, itd.

Kod aplikacji przygotuj do nagrania na płytę. W folderze konsolowa powinno znaleźć się archiwum całego projektu o nazwie *konsola.zip*, skopiowany z projektu plik z kodem źródłowym programu oraz plik wykonywalny, jeżeli istnieje.

Część II. Aplikacja webowa

Za pomocą środowiska programistycznego dostępnego na stanowisku egzaminacyjnym, wykonaj aplikację webową która umożliwi dodawanie wpisów gościom którzy odwiedzili stronę. Do zbudowania aplikacji wykorzystaj bazę danych z archiwum **pliki4.7z** zabezpieczonego hasłem: **Gu3stB00k!**

The screenshot shows a web application titled "Guestbook". It has two main sections. On the left, under the heading "Wpisy", there is a list of three entries, each in a light gray box. The first entry is "Pozdrowienia dla autora" by "Kasia", the second is "Warto tu zajrzeć" by "Robert", and the third is "Wszystko ok" by "Magda". On the right, under the heading "Dodaj", there is a form with three input fields: "Tytuł", "Wpis", and "Autor", and a blue "Dodaj" button.

Obraz 2. Aplikacja webowa, w stanie początkowym

Na obrazie 2 przedstawiono ideę aplikacji. W zależności od zastosowanego środowiska programistycznego wygląd może nieznacznie się różnić.

Opis wyglądu aplikacji

- Okno zawiera kontrolki rozmieszczone zgodnie z obrazem 2. Są to:
 - Wyśrodkowany nagłówek 1 stopnia o treści "Guestbook"
 - 2 kolumny z nagłówkami 3 stopnia o treści "Wpisy" oraz "Dodaj"
 - W 1 kolumnie znajduje się lista dodanych wpisów w postaci kart o szarym kolorze tła. Każda karta składa się z:
 - Pogrubionego tytułu
 - Treści wpisu
 - Podpisu użytkownika umieszczonego po prawej stronie, poprzedzonego tyldą
 - W 2 kolumnie znajduje się formularz do wprowadzania danych, składający się z wymaganych pól formularza:
 - Grupy zawierającej etykietę "Tytuł" oraz pole wprowadzania danych
 - Etykiety "Wpis"
 - Obszaru tekstowego do wprowadzania treści wpisu
 - Grupy zawierającej etykietę "Autor" oraz pole wprowadzania danych
 - Przycisku wysyłania formularza o treści "Dodaj"

Opis działania aplikacji

- Po uruchomieniu aplikacji jest wysyłane zapytanie o listę wszystkich wpisów, które są następnie wypisane w kolumnie "Wpisy"
- Po naciśnięciu przycisku „Dodaj” wysyłane jest zapytanie do serwera aby dodać wpis, po którym automatycznie wysyłamy ponownie zapytanie o listę wpisów

entries		
id	INT(11)	PK
title	VARCHAR(50)	NN
content	TEXT	NN
author	VARCHAR(50)	NN
created_at	TIMESTAMP	NN

Obraz 3. Schemat bazy danych guestbook

Założenia API:

- Działa na porcie ustalonym w pliku zmiennych środowiskowych, inaczej 3000
- Dane do połączenia z bazą są przechowywane w pliku zmiennych środowiskowych
- Baza danych *guestbook* pochodzi z archiwum dołączonego do arkusza*.
- Posiada 3 ścieżki:
 - ścieżkę */entries* (GET) – odczytującą z bazy danych wpisy i zwracającą je jako JSON
 - ścieżkę */entry* (POST) – odczytującą dane do niej przekazane i dodaje wiersz do bazy
 - ścieżkę */entry/<NUMER ID>* (DELETE) – usuwającą wpis o podanym identyfikatorze
- ścieżka dodająca wpis sprawdza, czy przekazany JSON zawiera wszystkie wymagane dane. W przeciwnym razie zwraca komunikat błędu z odpowiednim kodem statusu HTTP.
- ścieżka usuwająca sprawdza przed usunięciem czy użytkownik ma takie uprawnienia w nagłówku zapytania ("*x-api-key*"). Klucz *x-api-key* jest zapisany w pliku zmiennych środowiskowych. Jeżeli klucz jest poprawny, wpis zostaje usunięty z bazy.
- Każda odpowiedź zapytania jest w języku polskim lub angielskim i dokładnie opisuje, co zostało wykonane.
- API posiada middleware, który loguje do konsoli datę i czas (np. w formacie ISO), metodę HTTP oraz ścieżkę żądania.

*Obraz 3 przedstawia bazę MySQL, w przypadku MongoDB *id* ma inną nazwę: *_id*

Założenia aplikacji

- Aplikacja komunikuje się z zapleczem poprzez API
- Do aplikacji został użyty Bootstrap, bez użycia własnego CSS-a.
- Kod aplikacji powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania
- Należy stosować znaczące nazwy zmiennych, metod i kontrolek
- Nie jest wymagana walidacja danych po stronie klienta, zakładamy że są podane wszystkie dane prawidłowo.

Podejmij próbę kompilacji i uruchomienia aplikacji. Wykonaj zrzut ekranowy stanu początkowego aplikacji i stanu po dodaniu nowego wpisu (uwzględnij logi middleware widoczne w terminalu po dodaniu wpisu). Zrzuty ekranu zapisz w folderze webowa pod nazwami: *webowa1.png*, *webowa2.png*

Kod aplikacji przygotuj do nagrania na płytę. W podfolderze webowa należy utworzyć katalog frontend i backend (jeśli wymaga tego projekt). W katalogu *frontend/backend* powinno znaleźć się archiwum całego projektu o nazwie *frontend.zip/backend.zip* oraz plik (lub pliki) z kodem źródłowym modyfikowanym w czasie egzaminu.

Część III. Dokumentacja aplikacji

Wykonaj dokumentację do aplikacji utworzonych na egzaminie. W edytorze tekstu pakietu biurowego utwórz plik z dokumentacją i nazwij go *egzamin*. Dokument powinien zawierać podpisane zrzuty ekranu, a następnie zapisane informacje:

- nazwę systemu operacyjnego, na którym pracował zdający,
- nazwy środowisk programistycznych, z których zdający korzystał na egzaminie,
- nazwy języków programowania użytych podczas tworzenia aplikacji,
- opcjonalnie komentarz do wykonanej pracy.

dokument umieść w folderze o nazwie *dokumentacja*.

Tabela 1. Selected HTTP status codes

Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
409	Conflict

Tabela 2. Sending GET & POST requests in JavaScript

To communicate with the server, use the *fetch()* method with the endpoint URL. For GET requests, simply provide the URL to retrieve data from the server. For POST requests, include an options object specifying the method as "POST", set the *Content-Type* header to *application/json*, and pass the data as a JSON string in the request body using *JSON.stringify()*. After receiving the response, convert it to JSON format using the *.json()* method. Always wrap requests in a try-catch block to handle potential errors and log error messages to the console.

UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować podfoldery: konsola, webowa, testy i plik egzamin. W folderze konsola: cały projekt aplikacji konsolowej i zrzuty. W folderze webowa: cały projekt aplikacji webowej i zrzuty. W folderze testy: cały kod testów/archiwum aplikacji testującej, ewentualnie inne przygotowane pliki. Po nagraniu płyty sprawdź poprawność nagrania. Opisz płytę numerem zdającego i pozostaw na stanowisku, zapakowaną w pudełku wraz z arkuszem egzaminacyjnym.

Czas przeznaczony na wykonanie zadania wynosi 180 minut.

Ocenie będą podlegać 4 rezultaty:

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja webowa,
- dokumentacja aplikacji